

Organización del Proyecto

Para que todos puedan avanzar en paralelo desde el primer día, lo más importante es que en la primera reunión se defina el **contrato de interfaz**: rutas de la API, formatos JSON y variables de entorno. Con eso claro, cada integrante puede trabajar sin depender de que otro termine primero.

Distribución de Roles (Equipo de 5)

Integrante	Rol Oficial	Responsabilidad Principal	Entregable Concreto
1	Infraestructura & Gateway	Provisionar la VM Ubuntu Server en VirtualBox, configurar red Host-Guest, Nginx como reverse proxy y servir el build de Vue.	Script/instrucciones de la VM y archivo <code>nginx.conf</code> probado con endpoints dummy.
2	Datos & Persistencia	Contenedor PostgreSQL único con volúmenes, 1 BD por microservicio, esquemas DDL y scripts de inicialización con carnés.	Scripts SQL (<code>init.sql</code> , migraciones) y documentación de variables de conexión.
3	Backend de Dominio	Microservicios Flask (catálogo, inventario, clientes, pedidos, reportes) con lógica transaccional de stock atómico.	Código Flask de cada servicio + endpoints listos con datos mock o BD local.
4	Frontend (Vue)	Vistas en Vue: CRUD de catálogo/inventario, creación de pedidos, dashboard con alertas y visualización de carnés.	Carpeta del proyecto Vue capaz de generar el build de producción (<code>dist/</code>).
5	DevOps, Costos & Doc	Orquestación con <code>docker-compose.yml</code> , Dockerfiles, subida a Docker Hub, anexo de costos (CAPEX vs OPEX) y diapositivas.	<code>docker-compose.yml</code> integrador, repositorio Docker Hub con imágenes y reporte de costos.

Cómo trabajar en paralelo sin bloquearse

- **Frontend vs Backend:** Definir en la reunión inicial el formato JSON de los endpoints (ej. `GET /api/inventario/productos`). El frontend puede usar datos falsos (mock data) mientras el backend se termina.

- **Backend vs Base de Datos:** El backend puede levantar un contenedor genérico de Postgres y trabajar con esquemas preliminares, mientras el responsable de Datos afina las bases y seeds.
- **DevOps vs Código:** El encargado de Compose puede armar `docker-compose.yml` y Dockerfiles con aplicaciones mínimas (“Hello World”) sin esperar el código final.
- **Infraestructura vs Todo:** La VM se puede configurar de forma aislada (Ubuntu Server, Docker Engine, hostname, reglas de red en VirtualBox).

Agenda de la Primera Reunión (30–45 min)

1. Acordar variables de entorno (`.env`): nombres de las 5 bases de datos, puertos internos, usuarios y contraseñas.
2. Definir nombres de servicios en la red de Docker (ej. `db` , `api-inventario` , `api-pedidos` , `api-catalogo` , `api-clientes` , `api-reportes` , `gateway`).
3. Fijar el contrato de API para la transacción crítica: estructura del payload de `POST /api/pedidos` (productos, cantidades, carné) y respuesta en caso de stock insuficiente.
4. Recopilar los carnés de los 5 integrantes para los scripts de carga inicial.

Lista de verificación para la defensa

- Evidencias obligatorias (evitar -4 pts): hostname de la VM con identificador del grupo, carné visible en la UI y tabla de pedidos, capturas con reloj visible y URL de Docker Hub.
- Flujo de la demo (12 min): probar previamente `docker compose down` seguido de `docker compose up` para demostrar persistencia de datos en el volumen de Postgres.
- Preguntas cruzadas: dedicar 15 minutos a que cada integrante explique brevemente su archivo al resto del grupo, ya que el docente puede preguntar cualquier parte.